

project_plan

项目计划书：电荷密度的并行输出与输入优化（ABACUS）

1. 项目基本信息

- 项目名称：电荷密度 Cube/Rhog I/O 并行化与性能优化
- 小组成员：钟煜晨、查皓轩、徐子恒
- 项目背景：ABACUS 在自洽场（SCF）结束后需要输出三维电荷密度/静电势（Cube），以及在 `init_chg = "file"` 时读取电荷密度（`rhog/cube`）。现有实现存在明显性能瓶颈：Pool 归约同步阻塞、rank 0 串行读写、数据重排访存不友好、文本格式化开销大，导致大体系与频繁输出场景下 I/O 成为主要瓶颈。

2. 背景与问题定义

2.1 电荷密度输出/输入的重要性

- 结果分析与可视化：VESTA、AtomEye 等软件需要 Cube 格式网格数据。
- 后续计算与继续运行：读取已有电荷密度用于恢复或作为后续计算输入。
- 瓶颈现象：体系越大、输出越频繁，I/O 与通信开销占比越高，降低并行效率。

2.2 现有流程与瓶颈定位（基于参考文档）

典型 Cube 输出调用链：

```
1  Charge::save_rho() 或同类输出函数
2      ↓
3  ModuleIO::write_vdata_palgrid()  【主要瓶颈点】
4      ↓
5  pgrid.reduce()  【MPI 归约操作】
6      ↓
7  write_cube()  【文件写入】
```

主要瓶颈：

瓶颈类型	位置/环节	问题描述
MPI 同步阻塞	<code>write_cube.cpp</code>	<code>MPI_Barrier</code> 导致等待，降低并行效率
串行 I/O	<code>write_cube.cpp</code>	仅 rank 0 写文件，其他进程空闲
数据重排	<code>charge_mpi.cpp</code>	三层循环访存不连续，cache 命中率低

瓶颈类型	位置/环节	问题描述
格式化开销	<code>write_cube.cpp</code>	文本格式输出带来较大字符串/格式化成本
读取同类瓶颈	<code>rhoq_io.cpp</code> 、 <code>read_cube.cpp</code>	串行读取 + 广播、插值 $O(n^3)$ 、广播大数组

3. 项目目标与范围

3.1 总体目标

- 在**保证数值正确与格式兼容**（Cube 可被常用可视化工具读取、rhog 与原实现一致）的前提下，显著降低“归约 + 写入/读取”耗时，提高并行效率与可扩展性。
- 形成可复用、可测试、可复现实验的优化实现，并提交完整报告与性能数据。

3.2 工作范围（题目选择与优先级）

必做（主线交付）

- 题目 1：MPI 归约操作的并行化**
 - 面向 `charge_mpi.cpp` 中 `reduce_diff_pools`：使用非阻塞通信（`MPI_Irecv / MPI_Isend`）替代阻塞通信，尝试实现计算与通信重叠。
- 题目 2：OpenMP 多线程加速数据重排**
 - 面向 `charge_mpi.cpp` 的三层重排循环：做循环依赖分析，采用 OpenMP 并行策略（如 `parallel for`、`collapse`）并改善访存模式。
- 题目 3：Cube 文件格式优化（并行 I/O）**
 - 面向 `write_cube.cpp`：使用 MPI-IO 实现并行写入；设计 file view/offset，避免仅 rank 0 串行写。

选做（加分项/时间允许时）

- 题目 6：电荷密度读取的 MPI-IO 并行化**（`rhoq_io.cpp` 的 `read_rhoq`）
- 题目 7：三线性插值的 OpenMP 并行化**（`read_cube.cpp` 的 `trilinear_interpolate`）

4. 技术路线与实现方案

4.1 基线评测与数据集

- 选用参考文档推荐体系建立统一基线（如 Si 64、Fe 128 等），记录：
 - 总时间与 I/O 占比（重点关注 `write_vdata_palgrid`、`pgrid.reduce`、`write_cube` 等）
 - 不同进程数（1/2/4/8/16）下的强扩展趋势
- 输出与读取分别建立基线数据，确保优化前后可对齐对比。

4.2 题目 1：Pool 归约并行化（MPI 非阻塞）

- 核心思路：
 - 将阻塞式 send/recv/同步等待替换为 `MPI_Irecv / MPI_Isend`，减少整体同步等待时间。
 - 将本地累加/重排等可并行计算穿插到通信推进阶段，提升重叠度。
- 正确性策略：
 - 与原实现对比：同一输入下逐元素比对（误差阈值根据数据范围设定，如 `1e-12` 或更严格）。
 - 多进程数一致性测试（1/2/4/8/16）。

4.3 题目 2：数据重排 OpenMP 并行化

- 核心思路：
 - 先做循环依赖分析，确保每次写入的输出索引唯一，避免竞争。
 - 采用合适的并行粒度（`collapse(2)` 等），并优先使用静态调度减少调度开销。
 - 尽量让最内层循环沿连续内存方向遍历，提高 cache 命中。
- 正确性策略：
 - 线程数变化下输出一致（1/2/4/8 线程）。
 - 覆盖边界条件：`ncxy=1`、`numz` 含 0/1、极小网格等。

4.4 题目 3：Cube 并行写（MPI-IO）

- 目标：在保持 Cube 头部与数据布局兼容的前提下，尽量实现并行写入，减少 rank 0 串行瓶颈。
- 关键技术点：
 - 计算每个进程对应数据块在文件数据区的 offset（必要时按 Cube 常见行格式进行分块处理）。
 - 使用 MPI-IO（如 `MPI_File_write_at_all`）协调并行写入，确保不冲突。
- 风险与备选方案：
 - 若文本 Cube 的 offset 计算与行格式约束导致复杂度过高，则采用折中策略（减少格式化开销、分段聚合写入）并在报告中明确权衡与原因。

4.5 单元测试与可测试性设计

- 测试覆盖：
 - 串行 vs 并行一致性
 - 不同进程数/线程数组合下的正确性
 - 边界条件与异常处理（文件不存在/网格不匹配等）
- 参考项目既有测试风格与框架（参考文档给出的测试入口位置）。

5. 里程碑与进度安排（阶段交付）

阶段 A：调研与基线建立

- 跑通推荐体系，获得优化前基线数据
- 明确主要热点（归约/重排/写入/读取各自耗时）

阶段 B：MPI + OpenMP 优化（题目 1 + 2）

- 完成非阻塞 MPI 归约实现并通过一致性测试
- 完成 OpenMP 重排并通过线程一致性测试

阶段 C：并行 I/O（题目 3）

- 完成 Cube 并行写（或折中方案）并验证格式兼容
- 建立端到端性能对比数据

阶段 D：性能评测与报告收尾

- 强扩展测试（1/2/4/8/16），绘制曲线并分析
- 输出最终报告（问题分析、方案、实现、测试数据与结论）

6. 小组分工（建议）

- 钟煜晨**：题目 1（MPI 非阻塞归约）、多进程正确性对比、通信性能评测
- 查皓轩**：题目 2（OpenMP 重排）、线程扩展测试与访存优化分析
- 徐子恒**：题目 3（并行 I/O）、测试框架整合、可视化/可读回兼容性验证与报告图表整理

协作机制：

- 每个模块至少进行 1 次交叉代码审阅（Code Review）
- 每次优化提交都附带最小可复现的测试与关键结果数据

7. 风险分析与应对

风险	影响	应对策略
文本 Cube 并行写 offset 难度较高	影响按期完成题目 3	预留折中方案：降低格式化成本 + 分段写，报告中说明取舍
并行累加顺序导致数值差异	影响正确性验收	采用严格对比阈值与确定性测试数据；必要时固定归约顺序或提升数值稳定性
运行环境（MPI/文件系统）差异	性能波动、可复现性下降	记录环境信息，至少在两套环境验证；提供脚本与参数确保复现

8. 验收标准（可量化）

8.1 正确性

- 并行实现结果与原实现一致（或误差在阈值内），覆盖多进程/多线程/边界条件
- 输出文件可被读取回并与内存数据一致；Cube 文件保持可视化工具可用（以课程要求为准）

8.2 性能

- 归约、重排、写入等关键热点耗时显著下降
- 强扩展曲线相较原实现有明显改善，I/O 等待时间占比降低

8.3 工程质量

- 代码结构清晰、可测试、可复现
- 提供性能测试脚本与测试说明，报告数据可核验

9. 交付物清单

- 优化后的源码（对应模块实现）
- 单元测试代码与运行说明
- 性能测试脚本与结果数据（含强扩展曲线）
- 项目报告（引言/现有代码分析/优化方案/性能测试/结论）